Manual Técnico - Sistema de Gestión de Drones

Información General del Proyecto

Nombre del Sistema

Sistema de Gestión de Drones

Descripción General

Este sistema es una aplicación web para la gestión y simulación de operaciones de drones. Permite cargar configuraciones desde archivos XML, decodificar mensajes de instrucciones, optimizar tiempos de ejecución, y visualizar estructuras mediante gráficos generados con Graphviz. Está diseñado para estudiantes y desarrolladores interesados en algoritmos de optimización y visualización de datos.

Objetivo del Proyecto

- Proporcionar una interfaz intuitiva para la administración de sistemas de drones.
- Implementar algoritmos de decodificación y optimización de mensajes.
- Generar visualizaciones gráficas de las estructuras y procesos.
- Facilitar la exportación de resultados en formato XML.

Alcance

El sistema cubre:

- Carga y validación de archivos XML con configuraciones de drones y mensajes.
- Decodificación de mensajes en instrucciones para drones.
- Simulación de tiempos óptimos de ejecución.
- Visualización de estructuras mediante Graphviz.
- Exportación de resultados optimizados.

Tecnologías Utilizadas

- **Lenguajes**: C# (.NET 10), HTML5, CSS3, JavaScript.
- **Frameworks**: ASP.NET Core (backend), Razor Pages (frontend), Bootstrap (UI).
- **Librerías**: jQuery, jQuery Validation, Viz.js (para renderizado de Graphviz en navegador), Graphviz (para generación de SVG).
- **Herramientas**: Visual Studio, .NET SDK, Graphviz.

Arquitectura del Sistema

Tipo de Arquitectura

Cliente-servidor con separación clara entre backend (API REST) y frontend (aplicación web). El backend maneja la lógica de negocio y procesamiento de datos, mientras el frontend proporciona la interfaz de usuario.

Diagramas

Diagrama de Componentes

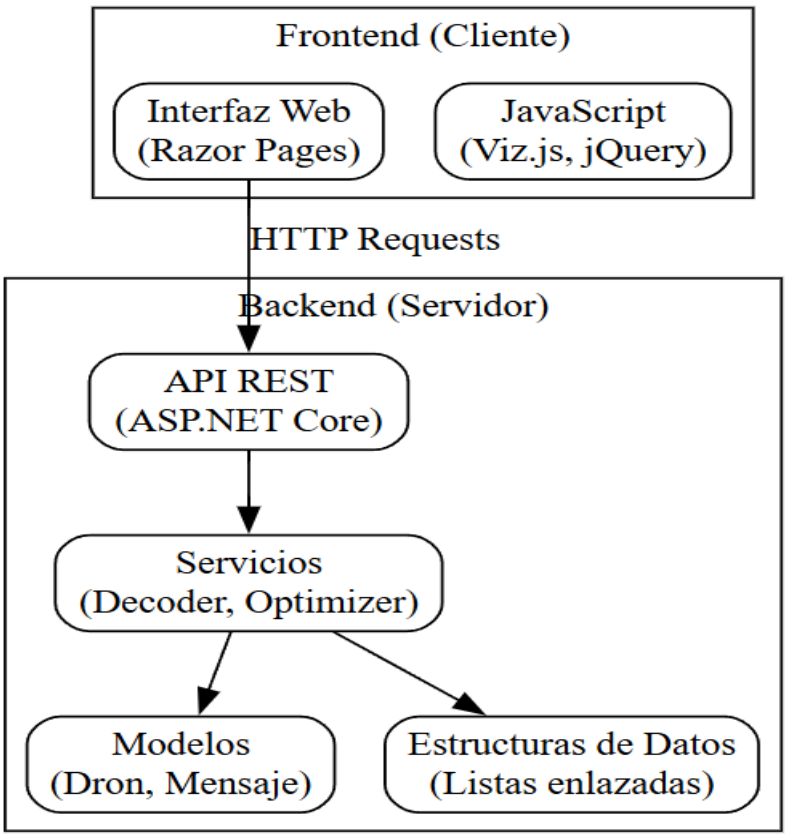
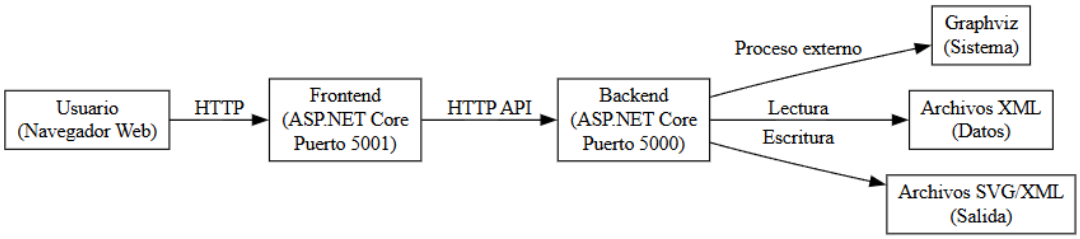


Diagrama de Despliegue



Descripción de Módulos Principales

Módulo de Procesamiento de Drones

- **Responsabilidad**: Gestiona la lógica de drones individuales y sistemas.
- **Componentes**: Clase `Dron` (altura, nombre), `SistemaDrones` (conjunto de drones).
- **Funciones**: Agregar drones, validar alturas, asignar a sistemas.

Módulo de Decodificación de Mensajes

- **Responsabilidad**: Convierte mensajes codificados en instrucciones ejecutables.
- **Componentes**: `DecoderService`, algoritmos de decodificación.
- **Funciones**: Procesar texto de mensajes, generar listas de instrucciones.

Módulo de Optimización

- **Responsabilidad**: Calcula tiempos óptimos de ejecución y simula procesos.
- **Componentes**: `OptimizerService`, algoritmos de optimización.
- **Funciones**: Simular tiempos reales, generar timelines.

Módulo de Carga XML

- **Responsabilidad**: Parsear y validar archivos XML de entrada.
- **Componentes**: `XMLReader`, validación de esquemas.
- **Funciones**: Cargar configuraciones, inicializar estructuras de datos.

Módulo de Visualización

- **Responsabilidad**: Generar gráficos DOT y convertir a SVG.
- **Componentes**: Integración con Graphviz, generación de código DOT.
- **Funciones**: Crear visualizaciones de estructuras y flujos.

Requisitos del Sistema

Hardware

- **RAM mínima**: 4 GB
- **Procesador**: Intel Core i3 o equivalente (2 núcleos, 2.5 GHz)
- **Espacio en disco**: 500 MB (para instalación y datos)

Software

- **Sistema operativo**: Windows 10/11, Linux (Ubuntu 18.04+), macOS (10.15+)
- **Versiones necesarias**:
- .NET SDK 10.0 o superior
- Graphviz 2.40 o superior (para generación de SVG)
- **Dependencias externas**: Navegador web moderno (Chrome, Firefox, Edge)

Instalación y Configuración

Pasos de Instalación

1. Clonar el repositorio: `git clone https://github.com/JimH0202/-IPC2-_-Proyecto-2_-202303768.git`
2. Instalar .NET SDK desde https://dotnet.microsoft.com/download
3. Instalar Graphviz desde https://graphviz.org/download/
4. Verificar instalación: `dotnet --version` y `dot -V`

Configuración de Variables de Entorno

- Asegurar que `dotnet` esté en PATH
- Asegurar que `dot` (Graphviz) esté en PATH
- Para Windows: Agregar `C:\Program Files (x86)\Graphviz\bin` a PATH

Instalación de Dependencias

- El proyecto usa NuGet para dependencias .NET (se resuelven automáticamente)
- No hay dependencias adicionales manuales

Cómo Ejecutar el Sistema

1. Abrir terminal en la raíz del proyecto
2. Ejecutar `.\run.bat` (Windows) o `./run.ps1` (PowerShell)
3. Abrir navegador en `http://localhost:5001` (frontend)
4. El backend corre en `http://localhost:5000`

Estructura del Proyecto

```
/IPC2-_-Proyecto-2_-202303768
├── -IPC2-_-Proyecto-2_-202303768.sln  # Solución Visual Studio
├── run.bat                            # Script de ejecución (Windows)
├── run.ps1                            # Script de ejecución (PowerShell)
├── Backend/                           # Proyecto backend
│   ├── Backend.csproj                 # Archivo de proyecto .NET
│   ├── Program.cs                     # Punto de entrada
│   ├── Controllers/                   # Controladores API
│   │   └── DroneSystemController.cs   # API principal
│   ├── Models/                        # Modelos de datos
│   │   ├── Dron.cs
│   │   ├── EstadoDron.cs
│   │   └── ...
│   ├── Services/                      # Servicios de negocio
│   │   ├── DecoderService.cs
│   │   ├── OptimizerService.cs
│   │   └── XMLReader.cs
│   ├── TDAs/                          # Estructuras de datos
│   │   ├── ListaAcciones.cs
│   │   ├── Nodo.cs
│   │   └── ...
│   └── bin/                           # Archivos compilados (generado)
├── Frontend/                          # Proyecto frontend
│   ├── Frontend.csproj                # Archivo de proyecto .NET
│   ├── Program.cs                     # Punto de entrada
│   ├── Pages/                         # Páginas Razor
│   │   ├── Index.cshtml
│   │   ├── Sistema.cshtml
│   │   └── ...
│   ├── Properties/                    # Configuraciones
│   └── wwwroot/                       # Archivos estáticos
│       ├── css/
│       ├── js/
│       └── lib/                       # Bootstrap, jQuery
├── Data/                              # Archivos de datos de salida
└── Graphviz/                          # Archivos DOT/SVG generados
```

Descripción de Componentes / Módulos

Clase Dron

- **Descripción**: Representa un dron individual con propiedades de altura y nombre.
- **Funciones principales**: Controlar altura actual, validar límites.
- **Entradas**: Nombre (string), altura máxima (int).
- **Salidas**: Estado del dron (altura actual, sistema asignado).

Clase Mensaje

- **Descripción**: Contiene instrucciones codificadas para drones.
- **Funciones principales**: Almacenar lista de instrucciones, asociar con sistema.
- **Entradas**: Nombre, sistema de drones, lista de instrucciones.
- **Salidas**: Texto decodificado, tiempo óptimo.

Parser XML (XMLReader)

- **Descripción**: Carga y valida archivos XML de configuración.
- **Funciones principales**: Parsear XML, inicializar estructuras de datos.
- **Entradas**: Ruta de archivo XML.
- **Salidas**: Objetos SistemaDrones, Mensaje, Dron poblados.

DecoderService

- **Descripción**: Decodifica mensajes en instrucciones ejecutables.
- **Funciones principales**: Procesar texto codificado, generar instrucciones.
- **Entradas**: Objeto Mensaje, SistemaDrones.
- **Salidas**: Lista de instrucciones (dron, altura).

OptimizerService

- **Descripción**: Calcula tiempos óptimos y simula ejecuciones.
- **Funciones principales**: Simular tiempos reales, optimizar secuencias.
- **Entradas**: Objeto Mensaje.
- **Salidas**: Tiempo óptimo (int), timeline de ejecución.

Interfaces y APIs

Endpoints Principales

- `POST /api/dronesystem/cargar-xml` : Carga archivo XML
- `GET /api/dronesystem/drones` : Lista drones
- `GET /api/dronesystem/sistemas` : Lista sistemas
- `GET /api/dronesystem/mensajes` : Lista mensajes
- `GET /api/dronesystem/mensaje/{nombre}` : Detalle de mensaje
- `GET /api/dronesystem/graphviz` : Genera visualización general
- `GET /api/dronesystem/mensaje/{nombre}/graphviz` : Visualización de mensaje
- `GET /api/dronesystem/exportar-xml` : Exporta resultados

Métodos

- POST: Para carga de archivos
- GET: Para consultas y generación de datos

Formato de Datos

- **Entrada**: FormData (para archivos), JSON (para parámetros)
- **Salida**: JSON (datos), XML (exportación)

Pruebas

Tipos de Pruebas

- **Unitarias**: Pruebas de clases individuales (Dron, Mensaje)
- **Integración**: Pruebas de servicios completos (DecoderService, OptimizerService)
- **Manuales**: Pruebas de UI y flujos completos

Cómo Ejecutarlas

- Para pruebas unitarias: `dotnet test` (si se agregan proyectos de test)
- Para pruebas manuales: Usar la interfaz web con archivos de prueba

Casos de Prueba Importantes

- Carga de XML válido
- Decodificación de mensaje simple
- Generación de visualización
- Exportación de resultados

Mantenimiento

Cómo Actualizar el Sistema

1. Actualizar .NET SDK si es necesario
2. Actualizar dependencias: `dotnet restore`
3. Compilar: `dotnet build`
4. Ejecutar pruebas

Cómo Agregar Nuevas Funcionalidades

1. Agregar modelos en `Models/`
2. Implementar lógica en `Services/`
3. Crear endpoints en `Controllers/`
4. Actualizar UI en `Pages/`

Manejo de Errores Comunes

1. **Error de carga XML**: Verificar formato del archivo
2. **Graphviz no encontrado**: Instalar y agregar a PATH
3. **Puerto ocupado**: Cambiar puertos en configuración

Seguridad

Manejo de Datos

1. Validación de entrada en todos los endpoints
2. Sanitización de nombres de archivos
3. Uso de tipos seguros en C#

Validaciones

- Verificación de existencia de archivos
- Validación de formatos XML
- Límites en tamaños de archivos

Encriptación

- Los mensajes se transmiten como texto plano (simulación)
- No hay encriptación implementada (ámbito educativo)

Ejemplos de Uso

Ejemplo de Entrada (XML)

```xml
<configuracion>
  <listaDrones>
    <dron>DRON001</dron>
  </listaDrones>
  <listaSistemasDrones>
    <sistemaDrones nombre="Demo">
      <alturaMaxima>100</alturaMaxima>
      <cantidadDrones>1</cantidadDrones>
      <contenido>
        <dron>DRON001</dron>
        <alturas><altura valor="50">L50</altura></alturas>
      </contenido>
    </sistemaDrones>
  </listaSistemasDrones>
  <listaMensajes>
    <Mensaje nombre="MENSAJE1">
      <sistemaDrones>Demo</sistemaDrones>
      <instrucciones>
        <instruccion dron="DRON001">50</instruccion>
      </instrucciones>
    </Mensaje>
  </listaMensajes>
</configuracion>
```

Ejemplo de Salida (Mensaje Decodificado)

- **Mensaje**: MENSAJE1
- **Sistema**: Demo
- **Decodificado**: DRON001 a 50m
- **Tiempo óptimo**: 5 segundos

Flujo del Sistema

1. Usuario carga XML → Sistema parsea y valida
2. Usuario selecciona mensaje → Se decodifica y optimiza
3. Usuario solicita visualización → Se genera DOT y SVG
4. Usuario exporta resultados → Se genera XML de salida

Problemas Conocidos

Limitaciones Actuales

- No soporta comunicación en tiempo real con drones
- Visualización requiere Graphviz instalado
- No hay persistencia de datos (solo archivos)

Bugs Conocidos

- En algunos casos, la visualización puede no renderizar si Viz.js falla
- Errores de decodificación en mensajes complejos pueden ocurrir

Referencias

Documentación Externa

- [ASP.NET Core Documentation](https://docs.microsoft.com/en-us/aspnet/core/)
- [Graphviz Documentation](https://graphviz.org/documentation/)

Librerías Usadas

- ASP.NET Core: Framework web de Microsoft
- Bootstrap: Framework CSS para UI
- jQuery: Librería JavaScript para manipulación DOM
- Viz.js: Renderizado de Graphviz en navegador

Enlaces Útiles

- Repositorio: https://github.com/JimH0202/-IPC2-_-Proyecto-2_-202303768
- .NET Download: https://dotnet.microsoft.com/download
- Graphviz Download: https://graphviz.org/download/

Diagramas en Graphviz

Diagrama de Clases

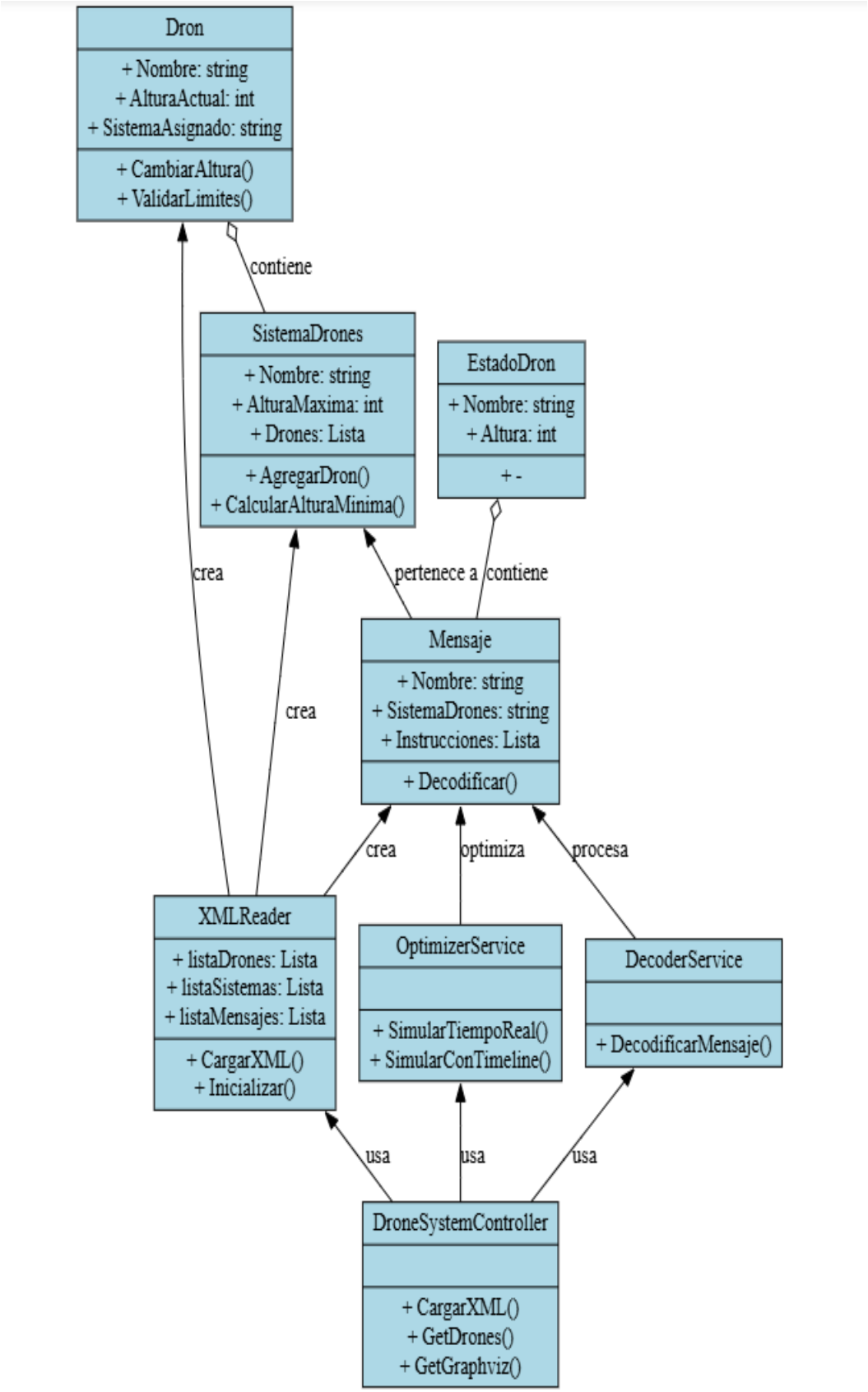
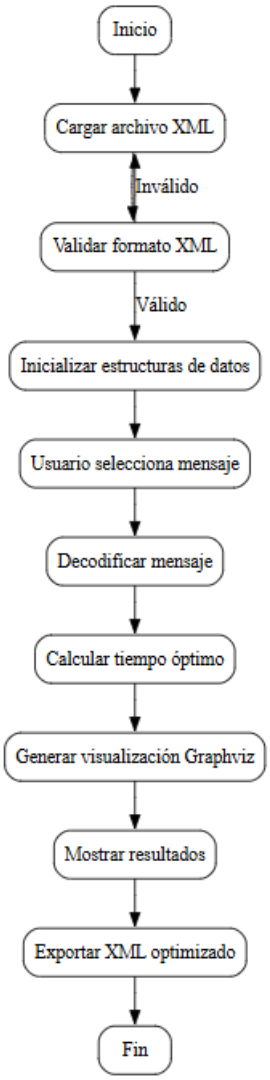
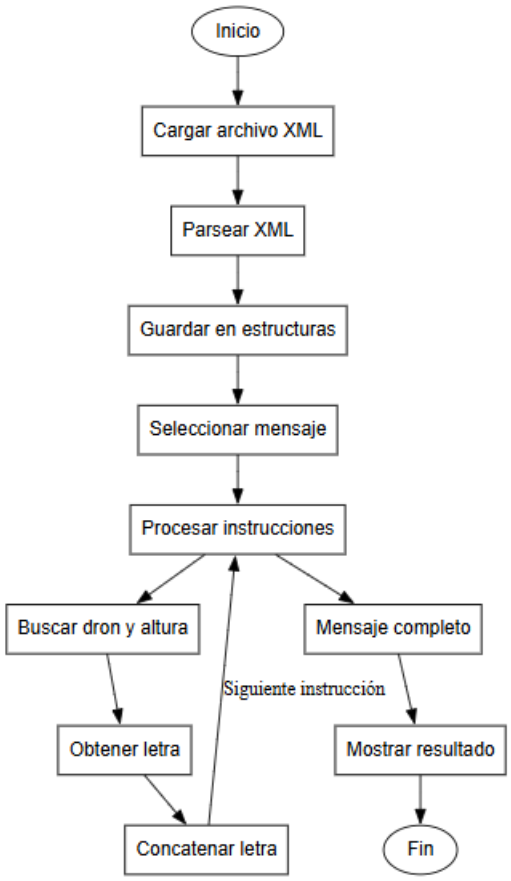
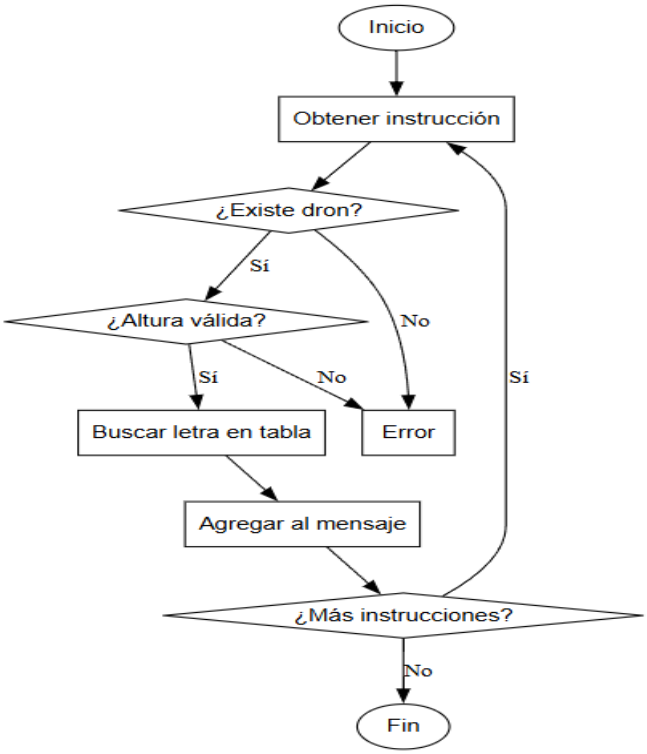



Diagrama de Flujo